# Polyfork for Unity

Offline documentation for version 0.12.x. Browse, remix and import 3D assets without leaving the editor.



---

## 1. Requirements and installation

Unity **6000.0 or newer**. Works with the built-in render pipeline and URP.

The package depends on two Unity Registry packages, which Unity resolves for you:

| Package | Why |
|---|---|
| `com.unity.cloud.gltfast` | Reads and writes the `.glb` files models arrive as |
| `com.unity.nuget.newtonsoft-json` | Parses the catalogue and each model's parameter schema |

Import the package, and Unity adds a **Polyfork** menu. An internet connection is required: the catalogue, the models and the rebuilds all come from polyfork.dev.

> **Install it one way, not two.** The `.unitypackage` and the git URL deliver the same files with the same GUIDs, so a project holding both ends up with two copies of every assembly and every native plugin, which will not compile. Remove one before adding the other.

---

## 2. The demo scene

Open `Demo/Polyfork Demo.unity` — under `Assets/Polyfork/` if you imported the `.unitypackage`, or under *Packages ▸ Polyfork for Unity* in the Project window if you installed from the git URL. It contains a camera, a key light, and an object named **START HERE - Polyfork**. Select it and the Inspector lists the four steps, with a button that opens the gallery.

> The scene ships without models on purpose. Polyfork is a browser for a catalogue that lives online, so what belongs in a demo scene is whatever *you* choose from it. Import one and it lands in this scene — that is the demo.

---

## 3. Browsing the catalogue

**Polyfork ▸ Browse Assets**, or `Ctrl/Cmd + Shift + P`.

- Search by name, and filter by kit, class, free-only, remixable-only or triangle budget.
- Click a card to select it: the panel on the right previews the real model, orbitable by dragging and zoomable by scrolling.
- The panel lists what the model can be changed into, without changing it.
- Paid models you do not own are dimmed and badged `locked`. They still preview and still remix; what needs a licence is writing one into your project.

---

## 4. Remixing a model

Press **Remix this asset**, or double-click a card. The model takes the whole window with its controls beside it. `Esc` returns to the catalogue.

| Control | What it does |
| --- | --- |

| | |
|---|---|
| Colourway chips | Recolours every part at once, instantly |
| Colour pickers | One vertex-colour slot each, instantly |
| Sliders | Reshapes the model — a rebuild |
| Dropdowns and toggles | Structural options — also a rebuild |

Every label, range, option and colour comes from the model's own published schema, so a parameter added on polyfork.dev appears here without updating this package.

---

## 5. Importing

**Import GLB to project** writes two files into `Assets/Polyfork/` — the `.glb` and a prefab beside it — and drops the prefab into the open scene in front of the camera, selected and undoable.

Models arrive at real-world scale with the origin on the ground, so they need no rescaling. Colour lives in the vertices on a single material: no textures to relink, no UV layout, and a set of models sharing that material merges into one draw call.

---

## 6. Editing a model after it is in the scene

The prefab carries a `PolyforkAssetLink` component holding the asset id and the parameter values. Select it and the Inspector shows the knobs again; move one and the model rebuilds where it stands.

Rebuilding replaces the meshes only, so the transform, children, colliders, any components you added and any material you assigned all survive the change.

---

## 7. Characters, rigs and animation

Rigged models import with an `Animator`, a `PolyforkCharacterAnimation` component and a set of clips bound to their own skeleton, **idle playing by default**. The Inspector has a dropdown for the rest.

From script:

```
var anim = character.GetComponent<PolyforkCharacterAnimation>();
anim.Play("walk");     // by name, case-insensitive
anim.Play(2);          // or by index
```

Characters animate *in place*: only rotation curves are bound, because position curves encode the source skeleton's proportions and would pull a differently-sized rig apart. That suits `applyRootMotion = false`, which the importer sets — you move the character, the clip poses it.

---

## 8. Instant local rebuilds

A parameter change can be rebuilt two ways. On the server it takes roughly 120 ms and counts against an hourly allowance. In the editor it is instant and costs nothing, because the editor runs the model's own program itself.

**Nothing needs installing.** The engine is QuickJS, via PuerTS, vendored into this package. Drag a slider and the geometry rebuilds as you drag.

It is editor-only and desktop-only: Windows, macOS (universal, so Apple Silicon is covered) and Linux on x64. The assembly is marked Editor-only and so is every native library, so none of it reaches a player build and a shipped game keeps using the server. **Polyfork ▸ Setup** reports whether the engine started and what to check if it did not.

> Rigged models always rebuild on the server, because the trimmed geometry library used locally omits the skinning classes.

> **Upgrading from 0.11 or earlier:** remove `com.tencent.puerts.core` and `com.tencent.puerts.quickjs` from the project. Unity refuses to import two native plugins with the same file name, so the project will not compile while both copies are present. Polyfork ▸ Setup detects this and says so.

---

## 9. Accounts, the free tier and API keys

No account is needed to browse, preview, remix or import free models — about half the catalogue — with no attribution and commercial use allowed.

An API key raises the hourly rebuild allowance and unlocks the paid catalogue. Add it under **Polyfork ▸ API Key…**. Keys are stored in `EditorPrefs` on your machine and never written into a scene or a repository. A key set through the `POLYFORK_API_KEY` environment variable or a `polyfork.key` file is picked up too, and the window says which source is in force.

---

## 10. Scripting reference

| Type | Purpose |
|---|---|
| `PolyforkCatalog` | Loads the catalogue and resolves models at run time |
| `PolyforkSpawner` | Spawns a model into a scene |
| `PolyforkRemixable` | Drives a spawned model's parameters at play time |
| `PolyforkAssetLink` | Records which model a GameObject is, and its values |
| `PolyforkCharacterAnimation` | Plays a chosen clip on a rigged model |

The **Runtime API** sample (Package Manager ▸ Samples) spawns a model and drives a parameter from script.

---

## 11. Troubleshooting

### The gallery is empty

Check the status bar at the bottom of the window; it reports what the last request did. An internet connection is required.

### Controls are greyed out

The hourly rebuild allowance is spent. Add an API key, or install a local engine (section 8) — a local rebuild is never metered, so nothing is greyed out once one is running.

### An imported model is grey

Colour lives in vertex colours. If you replace the material, use one that multiplies by vertex colour; Unity's Standard and URP/Lit shaders discard it.

### A character does not animate

Check the Console. The binding reports how many curves it matched; a very low number means the skeleton was not the expected one.

---

## 12. Licence and third-party notices

This package is MIT licensed. The full text is in `LICENSE.md`, and components with their own licences are listed in `Third Party Notices.md`, both included.

Models carry their own terms, stated on each model's page: free models allow commercial use with no attribution.

This documentation is also kept online, where it is updated between releases: [github.com/lucas-martinic/polyfork-unity-connector](github.com/lucas-martinic/polyfork-unity-connector). Support: [hello@polyfork.dev](mailto:hello@polyfork.dev).